



High-Performance Linpack (HPL)

Supercomputer benchmarking



Agenda

What is
computer
benchmarking?

The Top500 list

High-
Performance
Linpack

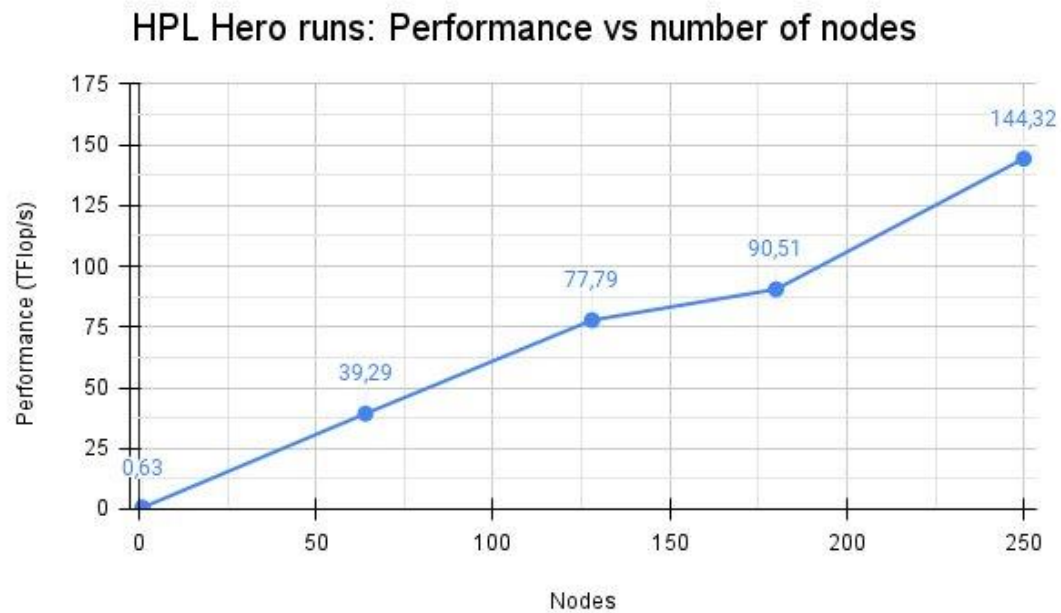
Computer benchmarking

1. What is it?
2. Why do we need to it?
3. Challenges in benchmarking
4. How to benchmark?



What is computer benchmarking?

- Testing the performance of a computer
- Trying to reach theoretical best performances
- Different possible aspects to benchmark
 - Processing power
 - Performance in half-precision
 - IO performance
 - Memory latency and bandwidth
 - Performance of specific applications (e.g. climate codes)
- How well the performance scales on cluster computers
 - Performance of the interconnect



Why benchmarking?

- Way to compare different computers and architectures
- Testing the computer under load
 - Fault tolerancy
 - Testing the cooling systems
- Testing the scaling of large clusters
 - Performance of the interconnect



Challenges and limitations in benchmarking

- How to thoroughly test and compare different hardware and architectures?
 - ARM vs. x86
- Performance depends also on the maturity of the software
- What parts of the computer to test?
 - CPU and GPU performance
 - Memory latency and bandwidth
 - IO capability
 - Extreme-scale usage
- Depending on the use-case, no single number can thoroughly describe a given computer system

How to benchmark a computer

- Choose what aspect you want to test
 - Processing power
 - IO
 - Your network connection?
- Download an appropriate software
 - Depends on your OS
 - HPL, HPCG, IO500 ...
 - A lot of commercial options available as well
- Run the benchmark
- Compare the results against similar systems



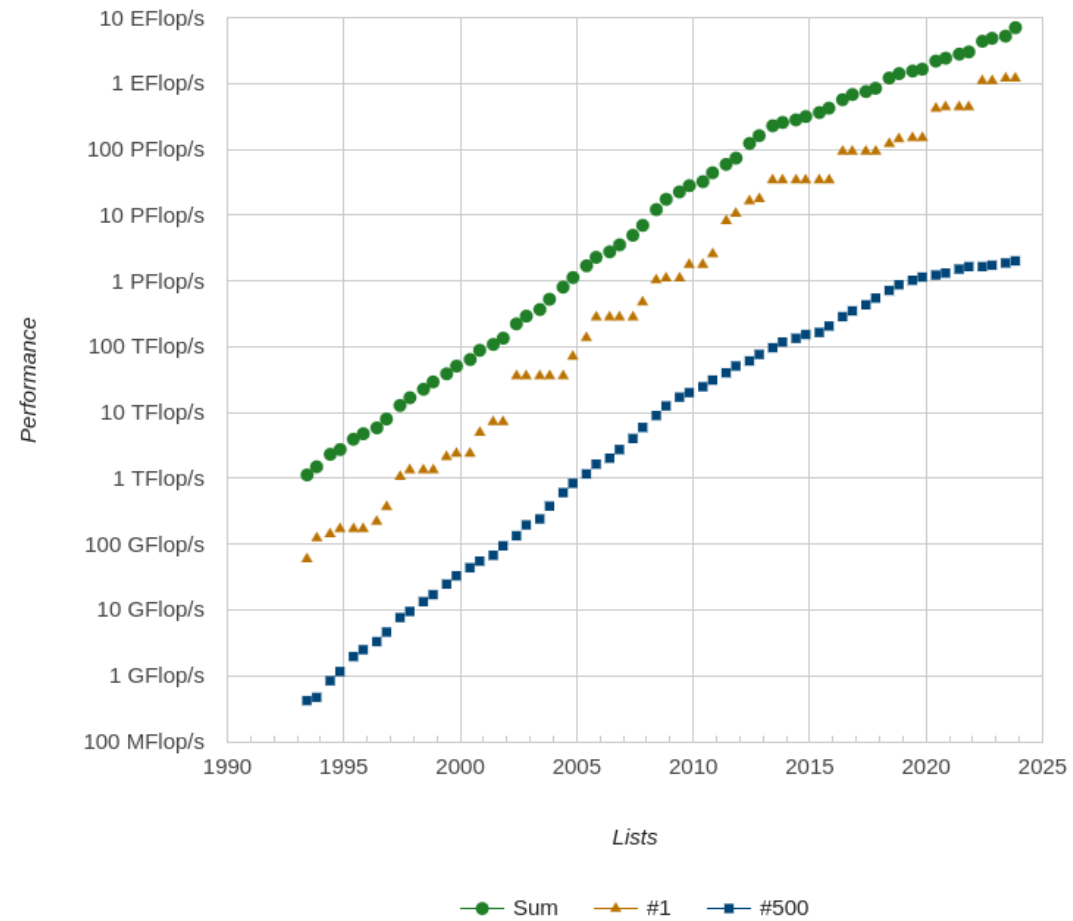
The Top500 list

The Top500 list

- Listing the 500 most powerful (known & commercially available) computer systems twice a year
- Main list is based on a single software that tries to maximize the "Flops" performance of the machine
 - High-performance Linpack
- Simplistic view of computer performance



Performance Development



Other metrics in the Top500 project

- HPL strains mainly the processors on modern supercomputers (GPUs/CPUUs)
- Doesn't tell the user much about a computer's IO and memory capabilities
 - Different applications might need this kind of performance
- Complementary lists that test other functionality of the computers
 - HPCG (top 10)
 - Green500
 - IO500
 - AI workloads



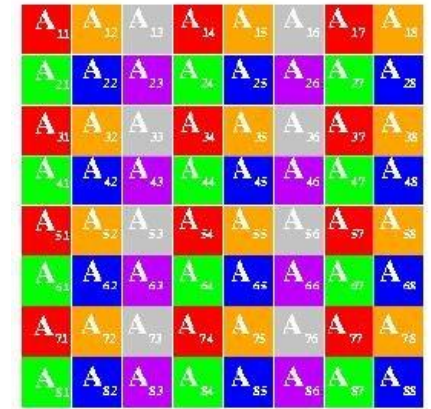
Regarding the student cluster competition

- Benchmarking section at the start of the competition
- Typically includes a number of benchmarks
 - HPL
 - HPCG
 - MLPerf Inference
- At IndySCC, only HPL is run
- Serves as a testing period before the main competition
 - Teams are graded based on this as well

High-Performance Linpack (HPL)

HPL - Basics

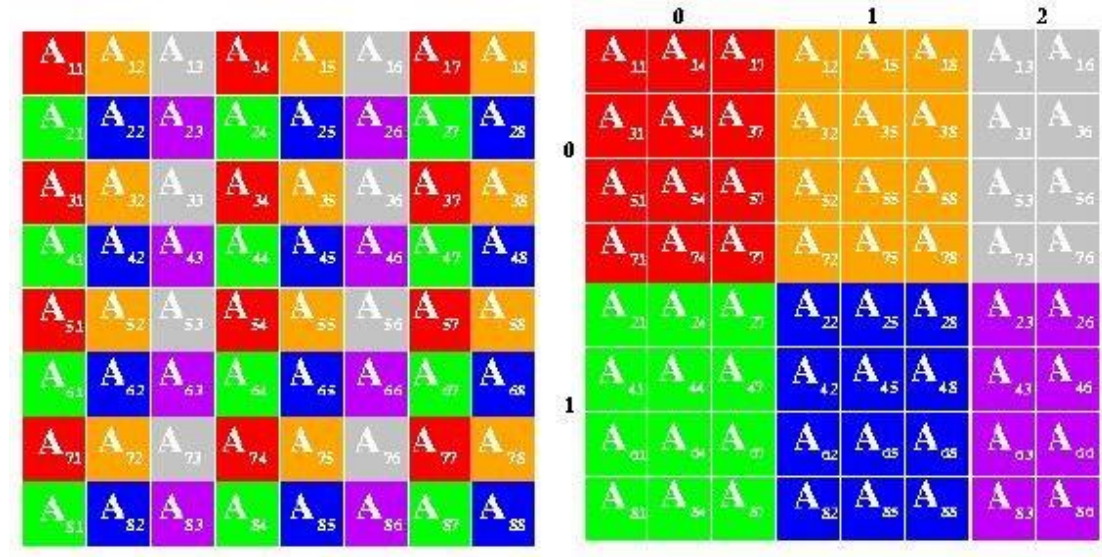
- The "main" Top500 list is based on results from HPL
- Solves a (random) dense linear system of $(Ax = b)$ in double precision
- The algorithm of the software is based in matrix multiplication
 - Results get close to the theoretical maximum performance of the hardware
- Great scaling on even very large clusters
- The algorithm can utilize GPUs well



$$\underbrace{\begin{bmatrix} 1 & 3 & 1 \\ 2 & 2 & 2 \\ 3 & 1 & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 5 \\ 6 \\ 5 \end{bmatrix}}_b$$

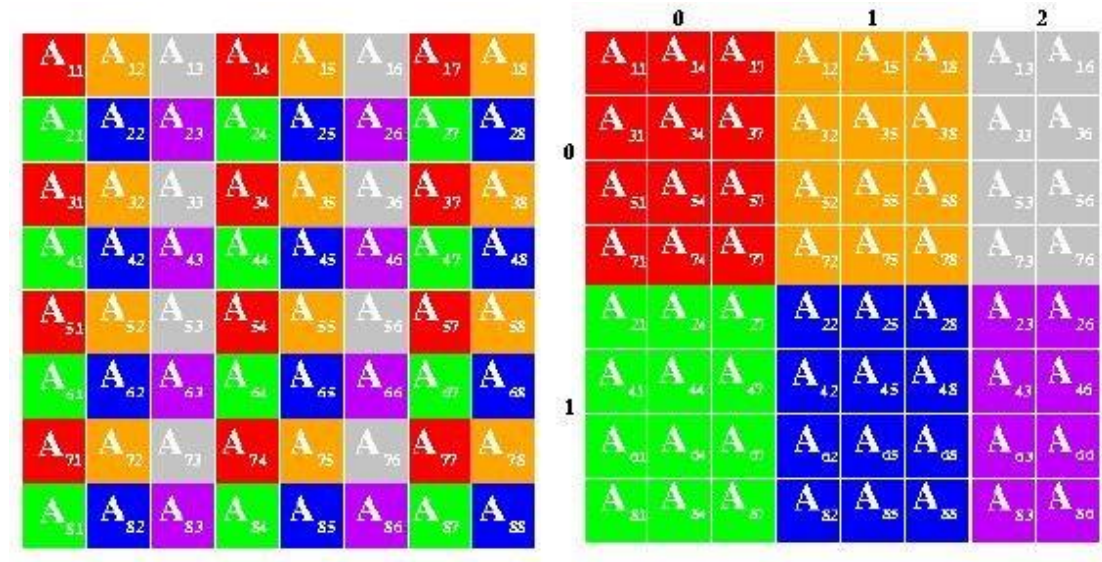
HPL – Parallelism

- The matrices are divided via a "block-cyclic distribution" into blocks of similar sizes
- These blocks can be given to processors to run
 - All individual blocks are solved before moving to the next step
 - The closer in size the blocks are, the better the load distribution
- P x Q Grid
- NB block size



HPL – Defining input values

- N: Size of the total matrix
- P x Q: Grid of the block-cyclic distribution
- NB: block size for sections



HPL – Installation

- Detailed instructions in today's "exercises" folder
- Fetching the source code
 - Multiple options available
 - We will use netlib's hpl today
 - <https://www.netlib.org/benchmark/hpl/>
- Defining our environment for the installation
 - What MPI are we using
 - Which linear algebra library are we specifying
 - Writing a Make file defining our environment

Running HPL

- Can be run with a "HPL.dat" input file
 - Size of the total matrix
 - Block sizes
 - How many total MPI processes
 - + A lot of fine-tuning options
- Running through Slurm
 - Make sure to reserve enough MPI processes (ntasks)

```
HPLinpack benchmark input file
Innovative Computing Laboratory, University of Tennessee
HPL.out      output file name (if any)
6            device out (6=stdout,7=stderr,file)
1            # of problems sizes (N)
460800      Ns
1            # of NBs
192          NBs
1            PMAP process mapping (0=Row-,1=Column-major)
1            # of process grids (P x Q)
16           Ps
8            Qs
16.0         threshold
1            # of panel fact
2            PFACTs (0=left, 1=Crout, 2=Right)
1            # of recursive stopping criterion
2            NBMINS (>= 1)
1            # of panels in recursion
2            NDIVs
1            # of recursive panel fact.
1            RFACTs (0=left, 1=Crout, 2=Right)
1            # of broadcast
0            BCASTs (0=1rg,1=1rM,2=2rg,3=2rM,4=Lng,5=LnM)
1            # of lookahead depth
0            DEPTHS (>=0)
2            SWAP (0=bin-exch,1=long,2=mix)
192          swapping threshold
1            L1 in (0=transposed,1=no-transposed) form
1            U in (0=transposed,1=no-transposed) form
0            Equilibration (0=no,1=yes)
8            memory alignment in double (> 0)
```

Getting started

- Instructions in the "exercises" folder
 - Install HPL on Mahti
 - Run HPL on Mahti on a single node
 - Run HPL with process counts of [1, 4, 16, 64]
 - Save your work in our github, under the deliverables folder
- Working in pairs
 - Same pairs as before

Tips for getting good performance with HPL

- Aim for a high memory usage (~80% of max memory)
 - Choose it to be divisible by your blocking size
 - For reference: <https://docs.csc.fi/computing/systems-mahti/>
- Use an automatic HPL file generator
 - https://www.advancedclustering.com/act_kb/tune-hpl-dat-file/
- A good blocking size can usually be found between 64-512
- On Mahti, the theoretical maximum for a single node is around 2500 GFlop/s



Thank you!



facebook.com/CSCfi



twitter.com/CSCfi



linkedin.com/company/csc--it-center-for-science



github.com/CSCfi